Spring Boot URL Shortener - Developer's Approach Guide

Table of Contents

1. Project Overview
2. Understanding the Architecture
3. Step-by-Step Developer Approach
4. Key Components Deep Dive
5. Development Workflow
6. Common Spring Boot Patterns Used
7. Database Design
8. Security Implementation
9. API Design
10. Testing Strategy
11. Deployment Considerations
12. Troubleshooting Guide

Project Overview

This is a **URL Shortener** application built with Spring Boot 3.4.0, similar to bit.ly or tinyurl.com. It allows users to: - Register and authenticate - Create shortened URLs from long URLs - Track click

analytics - View their URL history - Redirect users from short URLs to original URLs

Technology Stack

- **Backend**: Spring Boot 3.4.0 (Java 23)
- **Database**: PostgreSQL (with JPA/Hibernate)
- **Security**: Spring Security + JWT
- **Build Tool**: Maven
- **Additional Libraries**: Lombok, JJWT

Understanding the Architecture

1. Project Structure Analysis

```
src/main/java/com/url/shortener/
├── controller/           # REST API endpoints
├── dtos/                 # Data Transfer Objects
├── models/               # JPA entities
├── repository/           # Data access layer
├── security/             # Security configuration
├── service/              # Business logic layer
└── UrlShortenerSbApplication.java  # Main application class
```

2. Architecture Pattern

This project follows the **Layered Architecture** pattern: - **Controller Layer**: Handles HTTP requests/responses - **Service Layer**: Contains

business logic - **Repository Layer**: Data access abstraction - **Model Layer**: Entity definitions

---

Step-by-Step Developer Approach

Phase 1: Environment Setup & Understanding

1. **Clone and Inspect**
2. First thing: Read the `pom.xml` to understand dependencies
3. Check `application.properties` for configuration requirements
4. Look at the main application class to understand Spring Boot setup
5. **Database Setup** `bash # Set environment variables export DATABASE_URL=jdbc:postgresql://localhost:5432/urlshortener export DATABASE_USERNAME=your_username export DATABASE_PASSWORD=your_password export DATABASE_DIALECT=org.hibernate.dialect.PostgreSQLDialect export JWT_SECRET=your-secret-key export FRONTEND_URL=http://localhost:5173`
6. **Run the Application** `bash ./mvnw spring-boot:run`

Phase 2: Understanding the Domain Model

1. **Start with Entity Classes** (Always start here!)
2. `User.java` - Authentication and user management

3. `UrlMapping.java` - Core business entity
4. `ClickEvent.java` - Analytics tracking
5. **Understand Relationships**
6. User → UrlMapping (One-to-Many)
7. UrlMapping → ClickEvent (One-to-Many)

Phase 3: API Flow Analysis

1. **Authentication Flow**
2. POST `/api/auth/public/register` → User registration
3. POST `/api/auth/public/login` → JWT token generation
4. **Core Business Flow**
5. POST `/api/urls/shorten` → Create short URL
6. GET `/{shortUrl}` → Redirect to original URL
7. GET `/api/urls/myurls` → Get user's URLs
8. **Analytics Flow**
9. GET `/api/urls/analytics/{shortUrl}` → URL-specific analytics
10. GET `/api/urls/totalClicks` → User's total analytics

Phase 4: Security Understanding

1. **JWT Authentication**
2. Stateless authentication using JWT tokens
3. Token contains user information
4. Filter validates tokens on each request
5. **Authorization**

6. Method-level security with `@PreAuthorize`
7. Role-based access control (ROLE_USER)

Key Components Deep Dive

1. Models (Entity Layer)

User Entity

```
@Entity
@Table(name = "users")
public class User {
    private Long id;
    private String email;
    private String username;
    private String password;  // BCrypt hashed
    private String role;      // ROLE_USER
}
```

Developer Thinking: Simple user model with basic authentication fields. Password is hashed, role-based security implemented.

UrlMapping Entity

```
@Entity
public class UrlMapping {
    private Long id;
    private String originalUrl;    // The long URL
    private String shortUrl;       // 8-character random string
    private int clickCount;        // For basic analytics
    private LocalDateTime createdDate;
```

```
    @ManyToOne private User user;  // Owner relationship
    @OneToMany private List<ClickEvent> clickEvents;  // Detailed analytics
}
```

Developer Thinking: Core business entity. Short URL generation is simple (8 random chars). Both basic and detailed analytics supported.

2. Controllers (API Layer)

AuthController

- **Purpose**: Handle user registration and login
- **Endpoints**: `/api/auth/public/*` (public access)
- **Security**: No authentication required

UrlMappingController

- **Purpose**: Core URL shortening functionality
- **Security**: All endpoints require authentication
- **Key Methods**:
- `createShortUrl()` - Creates shortened URLs
- `getUserUrls()` - Lists user's URLs
- `getUrlAnalytics()` - Provides analytics data

RedirectController

- **Purpose**: Handle redirection from short to original URLs
- **Key Feature**: Updates click count and creates click events
- **Security**: Public access (no authentication needed)

3. Services (Business Logic Layer)

UrlMappingService

Core Algorithm - Short URL Generation:

```
private String generateShortUrl() {
    String characters = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz
    // Generate 8-character random string
    // Note: This is simple but has collision risk at scale
}
```

Developer Considerations: - ✅ Simple and fast - ⚠️ Potential collisions (no uniqueness check) - 🔄 Could be improved with collision detection

UserService

- Handles user registration with password hashing
- JWT token generation and validation
- User authentication logic

4. Security Configuration

WebSecurityConfig

Key Security Decisions:

```
.requestMatchers("/api/auth/**").permitAll()        // Public auth endpoints
.requestMatchers("/api/urls/**").authenticated()    // Protected URL operatio
.requestMatchers("/{shortUrl}").permitAll()         // Public redirect
```

JWT Filter Chain: 1. Request arrives 2. JWT filter extracts and validates token 3. Sets authentication context 4. Request proceeds to controller

---

Development Workflow

1. Starting Development (If building from scratch)

```
// 1. Start with Spring Boot setup
@SpringBootApplication
public class UrlShortenerApplication {
    public static void main(String[] args) {
        SpringApplication.run(UrlShortenerApplication.class, args);
    }
}

// 2. Define your domain model (entities first!)
// 3. Create repositories
// 4. Implement services
// 5. Build controllers
// 6. Add security
// 7. Test and refine
```

2. Adding New Features

Example: Adding URL expiration 1. **Model**: Add `expiryDate` field to `UrlMapping` 2. **Service**: Add expiration logic in `getOriginalUrl()` 3. **Controller**: Add expiration parameter to creation endpoint 4. **Repository**: Add method to find expired URLs 5. **Test**: Verify expiration behavior

3. Database Migrations

```
# Development
spring.jpa.hibernate.ddl-auto=update

# Production (recommended)
spring.jpa.hibernate.ddl-auto=validate
# Use Flyway or Liquibase for production migrations
```

Common Spring Boot Patterns Used

1. Dependency Injection

```
@RestController
@AllArgsConstructor  // Lombok generates constructor
public class UrlMappingController {
    private UrlMappingService urlMappingService;  // Injected
    private UserService userService;              // Injected
}
```

2. Data Transfer Objects (DTOs)

```
// Separates internal model from API contract
public class UrlMappingDTO {
    // Only fields that should be exposed via API
    private String originalUrl;
    private String shortUrl;
```

```
    private int clickCount;
    // No sensitive user data
}
```

3. Repository Pattern

```
public interface UrlMappingRepository extends JpaRepository<UrlMapping, Long>
    UrlMapping findByShortUrl(String shortUrl);
    List<UrlMapping> findByUser(User user);
}
```

4. Method-Level Security

```
@PreAuthorize("hasRole('USER')")
public ResponseEntity<UrlMappingDTO> createShortUrl(...) {
    // Only authenticated users can create URLs
}
```

Database Design

Entity Relationship Diagram

```
┌──────────────┐       ┌──────────────┐       ┌──────────────┐
│    User      │1     *│  UrlMapping  │1     *│ ClickEvent   │
│              │◄──────┤              │◄──────┤              │
│ - id         │       │ - id         │       │ - id         │
│ - username   │       │ - originalUrl│       │ - clickDate  │
│ - email      │       │ - shortUrl   │       │ - urlMapping│
```

```
| - password  |       | - clickCount |       └──────────────┘
| - role      |       | - createdDate|
└─────────────┘       | - user       |
                      └──────────────┘
```

Database Tables

```
-- Users table
CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    username VARCHAR(255) UNIQUE NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(50) DEFAULT 'ROLE_USER'
);

-- URL mappings table
CREATE TABLE url_mapping (
    id BIGSERIAL PRIMARY KEY,
    original_url TEXT NOT NULL,
    short_url VARCHAR(8) UNIQUE NOT NULL,
    click_count INTEGER DEFAULT 0,
    created_date TIMESTAMP NOT NULL,
    user_id BIGINT REFERENCES users(id)
);

-- Click events table (for analytics)
CREATE TABLE click_event (
    id BIGSERIAL PRIMARY KEY,
    click_date TIMESTAMP NOT NULL,
    url_mapping_id BIGINT REFERENCES url_mapping(id)
);
```

Security Implementation

1. Password Security

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();  // Industry standard
}
```

2. JWT Token Structure

```
{
  "sub": "username",
  "iat": 1640995200,
  "exp": 1641081600,
  "authorities": ["ROLE_USER"]
}
```

3. Security Filter Chain

```
HTTP Request
     ↓
CORS Filter
     ↓
JWT Authentication Filter  ← Custom filter
     ↓
Spring Security Filter Chain
     ↓
Controller Method
     ↓
```

```
@PreAuthorize Check
      ↓
Business Logic
```

API Design

1. RESTful Endpoints

```
Authentication:
POST /api/auth/public/register  - User registration
POST /api/auth/public/login     - User login

URL Operations:
POST /api/urls/shorten          - Create short URL
GET  /api/urls/myurls           - Get user's URLs
GET  /api/urls/analytics/{id}   - Get URL analytics
GET  /api/urls/totalClicks      - Get total clicks

Redirection:
GET  /{shortUrl}                - Redirect to original URL
```

2. Request/Response Examples

Create Short URL:

```
// Request
POST /api/urls/shorten
{
  "originalUrl": "https://www.example.com/very-long-url"
}
```

```
// Response
{
  "id": 1,
  "originalUrl": "https://www.example.com/very-long-url",
  "shortUrl": "AbC123Xy",
  "clickCount": 0,
  "createdDate": "2024-01-15T10:30:00",
  "username": "john_doe"
}
```

3. Error Handling

```
// Global exception handler recommended
@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(UrlNotFoundException.class)
    public ResponseEntity<?> handleUrlNotFound(UrlNotFoundException ex) {
        return ResponseEntity.notFound().build();
    }
}
```

Testing Strategy

1. Unit Testing

```
@ExtendWith(MockitoExtension.class)
class UrlMappingServiceTest {

    @Mock
```

```
    private UrlMappingRepository repository;

    @InjectMocks
    private UrlMappingService service;

    @Test
    void shouldCreateShortUrl() {
        // Test business logic in isolation
    }
}
```

2. Integration Testing

```
@SpringBootTest
@TestPropertySource(locations = "classpath:application-test.properties")
class UrlMappingControllerIntegrationTest {

    @Autowired
    private TestRestTemplate restTemplate;

    @Test
    void shouldCreateAndRetrieveShortUrl() {
        // Test full request/response cycle
    }
}
```

3. Repository Testing

```
@DataJpaTest
class UrlMappingRepositoryTest {

    @Autowired
```

```
    private TestEntityManager entityManager;

    @Autowired
    private UrlMappingRepository repository;

    @Test
    void shouldFindByShortUrl() {
        // Test database queries
    }
}
```

Deployment Considerations

1. Environment Configuration

```
# Development
spring.jpa.hibernate.ddl-auto=update
logging.level.org.springframework.security=DEBUG

# Production
spring.jpa.hibernate.ddl-auto=validate
logging.level.org.springframework.security=WARN
```

2. Docker Deployment

```
FROM openjdk:23-jdk-slim
COPY target/url-shortener-sb-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","/app.jar"]
```

3. Environment Variables

```
# Required for production
DATABASE_URL=jdbc:postgresql://prod-db:5432/urlshortener
DATABASE_USERNAME=prod_user
DATABASE_PASSWORD=secure_password
JWT_SECRET=very-secure-secret-key-for-production
FRONTEND_URL=https://yourfrontend.com
```

Troubleshooting Guide

1. Common Issues

Issue: "Unable to connect to database"

```
# Check environment variables
echo $DATABASE_URL
echo $DATABASE_USERNAME

# Verify database is running
pg_isready -h localhost -p 5432
```

Issue: "JWT token invalid"

```
# Check JWT secret is set
echo $JWT_SECRET

# Verify token format in request headers
Authorization: Bearer <your-jwt-token>
```

Issue: "Short URL collision"

```
// Current implementation doesn't check for collisions
// Improvement needed:
private String generateUniqueShortUrl() {
    String shortUrl;
    do {
        shortUrl = generateShortUrl();
    } while (urlMappingRepository.existsByShortUrl(shortUrl));
    return shortUrl;
}
```

2. Debugging Tips

Enable SQL Logging:

```
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
logging.level.org.hibernate.SQL=DEBUG
```

Security Debugging:

```
logging.level.org.springframework.security=DEBUG
```

Profile-based Configuration:

```
# Run with specific profile
java -jar app.jar --spring.profiles.active=dev
```

Performance Considerations

1. Database Optimization

- Add index on `short_url` column (unique constraint provides this)
- Add index on `user_id` for faster user URL lookups
- Consider partitioning `click_event` table by date

2. Caching Strategy

```
// Add caching for frequently accessed URLs
@Cacheable("shortUrls")
public UrlMapping getOriginalUrl(String shortUrl) {
    // Implementation
}
```

3. Short URL Generation Improvements

```
// Better approach: Use base62 encoding with incremental ID
private String generateShortUrl(Long id) {
    return Base62.encode(id);
}
```

Future Enhancements

1. Feature Additions

- URL expiration dates
- Custom short URL aliases
- QR code generation
- Bulk URL creation
- URL preview/safety checking

2. Technical Improvements

- Redis caching for popular URLs
- Rate limiting for API endpoints
- Database connection pooling
- Monitoring and metrics (Micrometer + Prometheus)
- API documentation (OpenAPI/Swagger)

3. Scalability Considerations

- Database sharding by user ID
- CDN for redirect endpoints
- Load balancing for multiple instances
- Asynchronous analytics processing

Developer Best Practices Applied

1. Code Organization

- ✅ Separation of concerns (Controller → Service → Repository)
- ✅ DTOs for API contracts
- ✅ Lombok for boilerplate reduction

2. Security

- ✅ Password hashing (BCrypt)
- ✅ JWT for stateless authentication
- ✅ Method-level security
- ✅ CORS configuration

3. Database Design

- ✅ Proper foreign key relationships
- ✅ Audit fields (createdDate)
- ✅ Separate analytics table for detailed tracking

4. Configuration

- ✅ Externalized configuration (environment variables)
- ✅ Profile-based settings
- ✅ Proper logging levels

Conclusion

This Spring Boot URL shortener demonstrates several key enterprise patterns: - **Clean Architecture**: Clear separation between layers - **Security First**: JWT authentication with role-based authorization - **Data Modeling**: Proper entity relationships and analytics tracking - **RESTful Design**: Well-structured API endpoints - **Configuration Management**: Environment-based configuration

Key Learning Points for New Spring Boot Developers: 1. Always start by understanding the domain model (entities) 2. Follow the dependency flow: Controller → Service → Repository 3. Security is configured once but affects all endpoints 4. DTOs protect your internal model from API changes 5. Environment-based configuration is crucial for deployment 6. Testing should cover all layers (unit, integration, repository)

Next Steps for Development: 1. Add comprehensive error handling 2. Implement caching for performance 3. Add API documentation 4. Create comprehensive test suite 5. Set up CI/CD pipeline 6. Add monitoring and logging

This project serves as an excellent foundation for understanding Spring Boot development patterns and can be extended with additional features as needed.